

Private Service Connect 介面 Vertex AI Pipelines

來源：<https://codelabs.developers.google.com/psc-interface-pipelines?hl=zh-tw>

步驟數：16

在本教學課程中，您將瞭解如何設定及驗證 Private Service Connect Vertex AI Pipelines

目錄

1. [簡介](#)
2. [事前準備](#)
3. [消費者設定](#)
4. [啟用 Identity-Aware Proxy \(IAP\)](#)
5. [建立消費者 VM 執行個體](#)
6. [Private Service Connect 網路連結](#)
7. [設定 Vertex AI Workbench](#)
8. [Vertex AI Pipelines 至 test-svc-1 的連線](#)
9. [更新 Vertex AI 服務代理](#)
10. [預設服務帳戶更新](#)
11. [部署 Vertex AI Pipelines 工作](#)
12. [驗證與 test-svc-1 的連線](#)
13. [Vertex AI Pipelines AI to test-svc-2 connectivity](#)
14. [驗證失敗，無法連線至 test-svc-2](#)
15. [清理](#)
16. [恭喜](#)

步驟 1 · 約 0 分鐘

Private Service Connect 介面 Vertex AI Pipelines

程式碼研究室簡介

subject上次更新時間：3月 27, 2026

account_circle作者：Deepak Michael, Rafa Carvalho

1. 簡介

Private Service Connect 介面是一種資源，可讓供應商虛擬私有雲 (VPC) 網路啟動與用戶網路中各種目的地的連線。供應商和用戶網路可以位於不同專案和機構中。

網路連結與 Private Service Connect 介面之間的連線，類似於 Private Service Connect 端點與服務連結之間的連線，但有兩項主要差異：

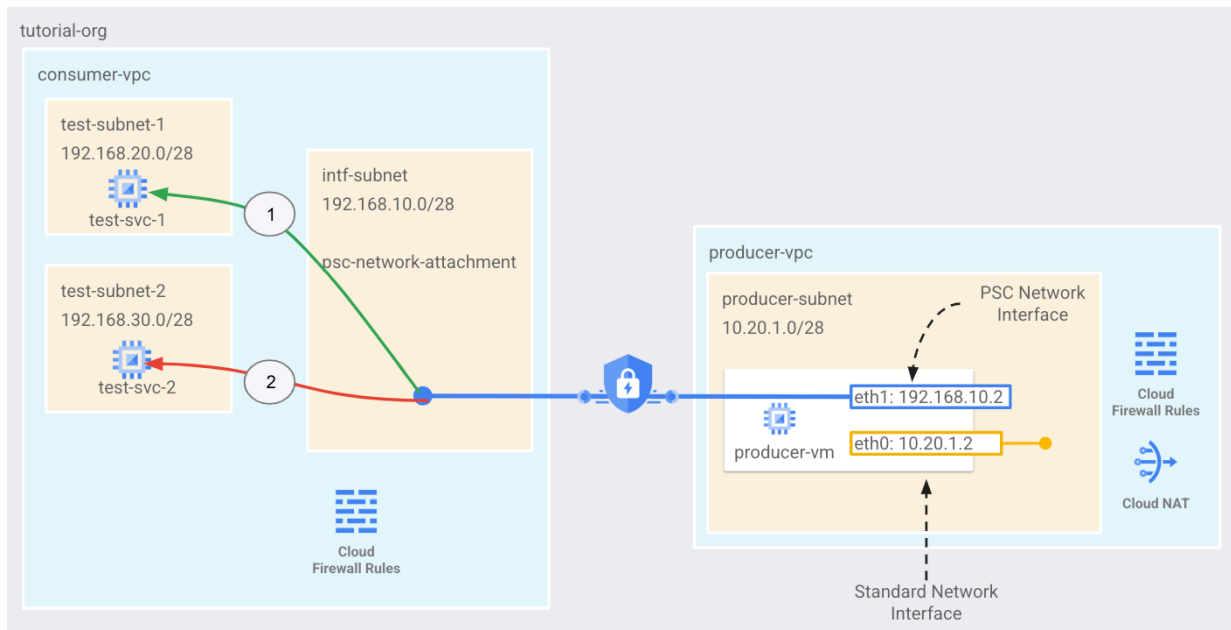
- 網路連結可讓供應商網路啟動與消費者網路的連線 (代管服務輸出)，而端點則可讓消費者網路啟動與供應商網路的連線 (代管服務輸入)。
- Private Service Connect 介面連線是可遷移的。也就是說，生產端網路可以與連線至消費端網路的其他網路通訊。

建構項目

Vertex AI Pipelines 會部署在 Google 代管的租戶專案中，並利用 PSC 網路附件在生產端和用戶網路之間建立多個 NIC 執行個體。由於 PSC 網路附件是透過用戶端網路的多重 NIC 部署，因此 Vertex AI Pipelines 可以連線至用戶端網路提供的路徑。

在本教學課程中，您將為 Vertex AI Pipelines 建構完整的 Private Service Connect (PSC) 介面架構，並使用 Cloud Firewall 規則允許或拒絕從生產者連線至消費者的測試執行個體，如圖 1 所示。

圖 1



您會在消費者 VPC 中建立單一 psc-network-attachment，產生下列用途：

- 在 consumer-vpc 中建立輸入防火牆規則，允許 Vertex AI Pipeline 子網路 (192.168.10.0/28) 連線至 test-svc-1。使用 TCPDUMP 確認從管道作業產生的 PING 是否成功傳送至 test-svc-1
- 在 consumer-vpc 中建立輸入防火牆規則，拒絕 Vertex AI Pipeline 子網路 (192.168.10.0/28) 連線至 test-svc-2。根據 Log Explorer 產生的防火牆記錄，確認 PING 失敗。

注意：本教學課程會根據圖示拓撲提供設定和驗證步驟，請視貴機構的需求修改程序。

課程內容

- 如何建立網路連結
- Vertex AI Pipelines 如何使用網路附件建立 PSC 介面
- 如何建立從製作人到消費者的通訊
- 如何允許從 Vertex AI Pipelines 存取消費者 VM test-svc-1
- 如何使用 Cloud Firewall，拒絕從 Vertex AI Pipelines 存取消費者 VM (test-svc-2)

軟硬體需求

- Google Cloud 專案
 - IAM 權限
 - [Compute 執行個體管理員](#) (roles/compute.instanceAdmin)
 - [Compute 網路管理員](#) (roles/compute.networkAdmin)
 - [Compute 安全管理員](#) (roles/compute.securityAdmin)
 - [受 IAP 保護的通道使用者](#) (roles/iap.tunnelResourceAccessor)
 - [Logging 管理員](#) (roles/logging.admin)
 - [Notebooks 管理員](#) (roles/notebooks.admin)
 - [專案 IAM 管理員](#) (roles/resourcemanager.projectIamAdmin)
 - [配額管理員](#) (roles/servicemanagement.quotaAdmin)
 - [服務帳戶管理員](#) (roles/iam.serviceAccountAdmin)
 - [服務帳戶使用者](#) (roles/iam.serviceAccountUser)
 - [Vertex AI 管理員](#) (roles/aiplatform.admin)
-

2. 事前準備

注意：除非生產者 (Vertex AI) 刪除已分配的資源，否則無法刪除網路附件。如要啟動刪除程序，請與 [Vertex AI 支援團隊](#) 聯絡。

本教學課程會使用 `$variables`，協助您在 Cloud Shell 中實作 `gcloud` 設定。

在 Cloud Shell 中執行下列操作：



```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

更新專案以支援教學課程

在 Cloud Shell 中執行下列操作：



```
gcloud services enable notebooks.googleapis.com
gcloud services enable aiplatform.googleapis.com
gcloud services enable compute.googleapis.com
gcloud services enable cloudresourcemanager.googleapis.com
```

步驟 3 · 約 10 分鐘

3. 消費者設定

建立消費者虛擬私有雲

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create consumer-vpc --project=$projectid --subnet-mode=custom
```

建立消費者子網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create test-subnet-1 --project=$projectid --range=192.168.20.0/28 --network=consumer-vpc --region=us-central1
```

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create test-subnet-2 --project=$projectid --range=192.168.30.0/28 --network=consumer-vpc --region=us-central1
```

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create workbench-subnet --project=$projectid --range=192.168.40.0/28 --network=consumer-vpc --region=us-central1 --enable-private-ip-google-access
```

Cloud Router 和 NAT 設定

由於筆記本執行個體沒有外部 IP 位址，因此本教學課程會使用 Cloud 網路位址轉譯 (NAT) 下載筆記本軟體套件。Cloud NAT 提供輸出 NAT 功能，也就是說，網際網路主機無法與使用者管理的 Notebook 啟動通訊，因此更加安全。

在 Cloud Shell 中，建立區域性 Cloud Router。

```
gcloud compute routers create cloud-router-us-central1 --network consumer-vpc --region us-central1
```

在 Cloud Shell 中，建立區域性 Cloud NAT 閘道。

```
gcloud compute routers nats create cloud-nat-us-central1 --router=cloud-router-us-central1 --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges --region us-central1
```

建立 Private Service Connect 網路連結子網路

在 Cloud Shell 中，建立 Vertex AI Pipelines 使用的網路附加元件子網路。

```
gcloud compute networks subnets create intf-subnet --project=$projectid --  
range=192.168.10.0/28 --network=consumer-vpc --region=us-central1
```

步驟 4 · 約 2 分鐘

4. 啟用 Identity-Aware Proxy (IAP)

如要允許 IAP 連線至您的 VM 執行個體，請根據以下條件建立防火牆規則：

- 套用至所有您希望能透過 IAP 存取的 VM 執行個體。
- 允許來自 IP 範圍 35.235.240.0/20 的輸入流量。這個範圍包含 IAP 用於 TCP 轉送的所有 IP 位址。

在 Cloud Shell 中，建立 IAP 防火牆規則。

```
gcloud compute firewall-rules create ssh-iap-consumer \  
  --network consumer-vpc \  
  --allow tcp:22 \  
  --source-ranges=35.235.240.0/20
```

步驟 5 · 約 3 分鐘

5. 建立消費者 VM 執行個體

在 Cloud Shell 中，建立取用者 VM 執行個體 test-svc-1。

```
gcloud compute instances create test-svc-1 \
  --project=$projectid \
  --machine-type=e2-micro \
  --image-family debian-11 \
  --no-address \
  --image-project debian-cloud \
  --zone us-central1-a \
  --subnet=test-subnet-1 \
  --shielded-secure-boot
```

在 Cloud Shell 中，建立取用者 VM 執行個體 test-svc-2。

```
gcloud compute instances create test-svc-2 \
  --project=$projectid \
  --machine-type=e2-micro \
  --image-family debian-11 \
  --no-address \
  --image-project debian-cloud \
  --zone us-central1-a \
  --subnet=test-subnet-2 \
  --shielded-secure-boot
```

取得並儲存執行個體的 IP 位址：

在 Cloud Shell 中，對測試 VM 執行個體執行說明。

```
gcloud compute instances describe test-svc-1 --zone=us-central1-a | grep networkIP:

gcloud compute instances describe test-svc-2 --zone=us-central1-a | grep networkIP:
```

範例：

```
user@cloudshell(psc-vertex)$ gcloud compute instances describe test-svc-1 --zone=us-central1-a | grep networkIP:

gcloud compute instances describe test-svc-2 --zone=us-central1-a | grep networkIP:
networkIP: 192.168.20.2
networkIP: 192.168.30.2
```

步驟 6 · 約 5 分鐘

6. Private Service Connect 網路連結

網路連結是區域資源，代表 Private Service Connect 介面的用戶端。您會將單一子網路與網路連結建立關聯，而生產者 (Vertex AI Pipelines) 會將 IP 指派給 Private Service Connect 介面。

建立網路連結

在 Cloud Shell 中建立網路連結。

```
gcloud compute network-attachments create psc-network-attachment \  
  --region=us-central1 \  
  --connection-preference=ACCEPT_MANUAL \  
  --subnets=intf-subnet
```

列出網路連結

在 Cloud Shell 中列出網路連結。

```
gcloud compute network-attachments list
```

說明網路連結

在 Cloud Shell 中，說明網路附件。

```
gcloud compute network-attachments describe psc-network-attachment --region=us-central1
```

請記下 psc-network-attachment URI，供應商建立 Private Service Connect 介面時會用到。

在下列範例中，psc 網路連結 URI 為：

```
projects/psc-vertex/regions/us-central1/networkAttachments/psc-network-attachment
```

```
user@cloudshell$ gcloud compute network-attachments describe psc-network-attachment --
region=us-central1
connectionPreference: ACCEPT_MANUAL
creationTimestamp: '2025-01-21T12:25:25.385-08:00'
fingerprint: m9bHc9gnosY=
id: '56224423547354202'
kind: compute#networkAttachment
name: psc-network-attachment
network: https://www.googleapis.com/compute/v1/projects/psc-vertex/global/networks/consumer-
vpc
region: https://www.googleapis.com/compute/v1/projects/psc-vertex/regions/us-central1
selfLink: https://www.googleapis.com/compute/v1/projects/psc-vertex/regions/us-
central1/networkAttachments/psc-network-attachment
subnetworks:
- https://www.googleapis.com/compute/v1/projects/psc-vertex/regions/us-
central1/subnetworks/intf-subnet
```

7. 設定 Vertex AI Workbench

下一節將引導您建立 Jupyter Notebook。這個筆記本將用於部署管道作業，從 Vertex AI Pipelines 將 PING 傳送至測試執行個體。Vertex AI Pipelines 與含有執行個體的消費者網路之間的資料路徑，會使用 Private Service Connect 網路介面。

建立使用者管理的服務帳戶

在下一節中，您將建立與本教學課程所用 Vertex AI Workbench 執行個體相關聯的服務帳戶。

在本教學課程中，服務帳戶會套用下列角色：

- [儲存空間管理員](#)
- [Vertex AI 使用者](#)
- [Artifact Registry 管理員](#)

在 Cloud Shell 中建立服務帳戶。

```
gcloud iam service-accounts create notebook-sa \  
  --display-name="notebook-sa"
```

在 Cloud Shell 中，將服務帳戶更新為 Storage 管理員角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/storage.admin"
```

在 Cloud Shell 中，使用 Vertex AI 使用者角色更新服務帳戶。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/aiplatform.user"
```

在 Cloud Shell 中，更新服務帳戶，並指派 Artifact Registry 管理員角色。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:notebook-  
sa@$projectid.iam.gserviceaccount.com" --role="roles/artifactregistry.admin"
```

在 Cloud Shell 中，允許筆記本服務帳戶使用 Compute Engine 預設服務帳戶，例項化 Pipeline Job。

```
gcloud iam service-accounts add-iam-policy-binding \  
  $(gcloud projects describe $(gcloud config get-value project) --  
format='value(projectNumber)')-compute@developer.gserviceaccount.com \  
  --member="serviceAccount:notebook-sa@$projectid.iam.gserviceaccount.com" \  
  --role="roles/iam.serviceAccountUser"
```

建立 Vertex AI Workbench 執行個體

在下一節中，建立納入先前建立的服務帳戶 notebook-sa 的 Vertex AI Workbench 執行個體。

在 Cloud Shell 中建立 private-client 執行個體。

```
gcloud workbench instances create workbench-tutorial --vm-image-project=deeplearning-  
platform-release --vm-image-family=common-cpu-notebooks --machine-type=n1-standard-4 --  
location=us-central1-a --subnet-region=us-central1 --subnet=workbench-subnet --disable-  
public-ip --shielded-secure-boot=true --service-account-email=notebook-  
sa@$projectid.iam.gserviceaccount.com
```

步驟 8 · 約 5 分鐘

8. Vertex AI Pipelines 至 test-svc-1 的連線

開啟新的 Cloud Shell 分頁，然後更新專案設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

如要允許從 Vertex AI Pipelines 連線至 test-svc-1，請建立輸入防火牆規則，將 PSC 網路附件指定為來源 (192.168.10.0/28)，並將 test-svc-1 IP 位址指定為目的地。

在 Cloud Shell 中，更新 destination-range，使其與 test-svc-1 IP 位址相符。

```
gcloud compute --project=$projectid firewall-rules create allow-icmp-vertex-pipelines-to-
test-svc1-vm --direction=INGRESS --priority=1000 --network=consumer-vpc --action=ALLOW --
source-ranges=192.168.10.0/28 --destination-ranges=<your-test-svc-1-vm-ip> --rules=icmp
```

範例：

```
gcloud compute --project=$projectid firewall-rules create allow-icmp-vertex-pipelines-to-
test-svc1-vm --direction=INGRESS --priority=1000 --network=consumer-vpc --action=ALLOW --
source-ranges=192.168.10.0/28 --destination-ranges=192.168.20.2 --rules=icmp
```

在 Cloud Shell 中使用 IAP 登入 test-svc-1 執行個體。

```
gcloud compute ssh test-svc-1 --project=$projectid --zone=us-central1-a --tunnel-through-iap
```

在作業系統中執行 tcpdump，擷取任何 icmp 流量。這個 OS 工作階段將用於驗證 Vertex AI Pipeline 與 VM 之間的通訊。

```
sudo tcpdump -i any icmp -nn
```

9. 更新 Vertex AI 服務代理

Vertex AI Pipelines 會代表您執行作業，例如從用於建立 PSC 介面的 PSC 網路連結子網路取得 IP 位址。為此，Vertex AI Pipelines 會使用[服務代理](#) (如下所列)，該代理需要[網路管理員](#)權限。

附註：更新服務代理權限前，請先前往 Cloud 控制台中的 Vertex AI，確認已啟用 Vertex AI API。

`service-$projectnumber@gcp-sa-aiplatform.iam.gserviceaccount.com`

在 Cloud Shell 中取得專案編號。

```
gcloud projects describe $projectid | grep projectNumber
```

範例：

```
gcloud projects describe $projectid | grep projectNumber:  
projectNumber: '795057945528'
```

在 Cloud Shell 中，使用 `compute.networkAdmin` 角色更新服務代理人帳戶。

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:service-<your-  
projectnumber>@gcp-sa-aiplatform.iam.gserviceaccount.com" --role="roles/compute.networkAdmin"
```

範例：

```
gcloud projects add-iam-policy-binding $projectid --member="serviceAccount:service-  
795057945528@gcp-sa-aiplatform.iam.gserviceaccount.com" --role="roles/compute.networkAdmin"
```

步驟 10 · 約 5 分鐘

10. 預設服務帳戶更新

啟用 Compute Engine API，並[授予預設服務帳戶](#) Vertex AI 存取權。請注意，存取權變更可能需要一段時間才會生效。

在 Cloud Shell 中，使用 aiplatform.user 角色更新預設服務帳戶

```
gcloud projects add-iam-policy-binding $projectid \  
  --member="serviceAccount:<your-projectnumber>-compute@developer.gserviceaccount.com" \  
  --role="roles/aiplatform.user"
```

範例：

```
gcloud projects add-iam-policy-binding $projectid \  
  --member="serviceAccount:795057945528-compute@developer.gserviceaccount.com" \  
  --role="roles/aiplatform.user"
```

11. 部署 Vertex AI Pipelines 工作

在下一節中，您將建立筆記本，對消費者測試專用服務 1 執行個體成功執行 PING。

在 Vertex AI Workbench 執行個體中執行訓練工作。

1. 在 Google Cloud 控制台中，前往 Vertex AI Workbench 頁面的執行個體分頁。
2. 按一下 Vertex AI Workbench 執行個體名稱 (workbench-tutorial) 旁的「Open JupyterLab」。Vertex AI Workbench 執行個體會在 JupyterLab 中開啟。
3. 依序選取「檔案」>「新增」>「筆記本」
4. 選取「Kernel」>「Python 3」
5. 在新的筆記本儲存格中執行下列指令，確認您擁有最新版本的 pip：

```
! pip3 install --upgrade --quiet google-cloud-aiplatform \
  kfp \
  google-cloud-pipeline-components
```

6. 在新筆記本儲存格中設定專案變數

```
PROJECT_ID = "<your-projectid>"
REGION = "<your-region>"
NETWORK_ATTACHMENT_NAME = "psc-network-attachment"
```

範例：

```
PROJECT_ID = "psc-vertex"
REGION = "us-central1"
NETWORK_ATTACHMENT_NAME = "psc-network-attachment"
```

7. 在新筆記本儲存格中，將全域不重複的 bucket 名稱定義為變數

```
BUCKET_URI = f"gs://<your-bucket-name>"
```

範例：

```
BUCKET_URI = f"gs://psc-vertex-bucket"
```

8. 在新的筆記本儲存格中建立值區

```
! gsutil mb -l {REGION} -p {PROJECT_ID} {BUCKET_URI}
```

在下一節中，您將決定要使用哪個預設的 Compute Engine 服務帳戶來執行管道作業，並授予適當的權限。

```

shell_output = ! gcloud projects describe $PROJECT_ID
PROJECT_NUMBER = shell_output[-1].split(":")[1].strip().replace("'", "")
SERVICE_ACCOUNT = f"{PROJECT_NUMBER}-compute@developer.gserviceaccount.com"

print(f"Project Number: {PROJECT_NUMBER}")
print(f"Service Account: {SERVICE_ACCOUNT}")

```

如要確認執行成功，系統會顯示服務帳戶和專案編號

9. 在新筆記本儲存格中，授予服務帳戶權限，以便讀取及寫入上一個步驟中建立的 bucket 管道構件。

```

! gsutil iam ch serviceAccount:{SERVICE_ACCOUNT}:roles/storage.objectCreator {BUCKET_URI}

! gsutil iam ch serviceAccount:{SERVICE_ACCOUNT}:roles/storage.objectViewer {BUCKET_URI}

```

10. 在新的筆記本儲存格中，定義管道參數。請注意，NETWORK_ATTACHMENT_NAME 是 PSC 網路連結，因此必須相符。

```

PIPELINE_ROOT = f"{BUCKET_URI}/pipeline_root/psc_test"
NETWORK_ATTACHMENT_URI =
f"projects/{PROJECT_NUMBER}/regions/{REGION}/networkAttachments/{NETWORK_ATTACHMENT_NAME}"

```

11. 在新的筆記本儲存格中，初始化 Vertex AI SDK

```

from kfp import dsl
from google.cloud import aiplatform, aiplatform_v1beta1
import time
from google.cloud.aiplatform_v1.types import pipeline_state
import yaml
from datetime import datetime
import logging
aiplatform.init(project=PROJECT_ID, location=REGION, staging_bucket=BUCKET_URI)

# Configure logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

```

12. 在新的筆記本儲存格中定義測試元件

```

@dsl.container_component
def ping_check(network_address: str):
    """Pings a network address

    Args:
        network_address: The IP address to ping
    """

    return dsl.ContainerSpec(
        image="ubuntu:22.04",
        command=["sh", "-c"],
        args=[
            f"""

            # Use sed for regex replacement, cleaner than bash parameter expansion for this
            cleaned_address=$(echo "{network_address}" | sed 's/[^0-9.]//g')
            apt-get update && apt-get install inetutils-traceroute inetutils-ping -y

            echo "Will ping $cleaned_address"
            if ! ping -c 3 $cleaned_address; then
                echo "Ping failed"
                traceroute -w 1 -m 7 $cleaned_address
                exit 1
            fi
            """
        ],
    )

```

13. 在新的筆記本儲存格中定義管道

```

@dsl.pipeline(name="check-connectivity")
def pipeline(ip_address: str):
    """Pings an IP address. Facilitated by a Private Service Connect Interface

    Args:
        ip_address: The IP address to ping
    """

    ping_check(network_address=ip_address).set_caching_options(False)
    return

```

14. 在新的筆記本儲存格中執行公用函式，等待管道完成作業

```

def wait_for_pipeline(
    project_id: str,
    region: str,
    pipeline_job_resource_name: str,
    timeout: int = 20 * 60, # Default timeout of 20 minutes (in seconds)
) -> bool:
    """
    Waits for a Vertex AI pipeline to finish, with a timeout.

    Args:
        project_id (str): The Google Cloud project ID.
        region (str): The region where the pipeline is running.
        pipeline_job_resource_name (str): The resource name of the pipeline job.
        timeout (int): The maximum time to wait for the pipeline to finish, in seconds.
                        Defaults to 20 minutes (1200 seconds).

    Returns:
        bool: True if the pipeline succeeded, False otherwise.

    Raises:
        TimeoutError: If the pipeline does not finish within the specified timeout.
    """

    # Initialize the AIPlatform client
    aiplatform.init(project=project_id, location=region)

    # Get the pipeline job
    pipeline_job = aiplatform.PipelineJob.get(resource_name=pipeline_job_resource_name)
    logging.info(
        f"Vertex AI Console Link: https://console.cloud.google.com/vertex-ai/pipelines/locations/{region}/runs/{pipeline_job.resource_name.split('/')[1]}?project={project_id}"
    )

    start_time = time.time()
    while True:
        status = pipeline_job.state
        logging.info(f"Pipeline Job status: {status.name}")

        if status in [
            pipeline_state.PipelineState.PIPELINE_STATE_SUCCEEDED,
            pipeline_state.PipelineState.PIPELINE_STATE_FAILED,
            pipeline_state.PipelineState.PIPELINE_STATE_CANCELLED,
        ]:
            break # Exit the loop if the job is finished

        if time.time() - start_time > timeout:
            logging.error(f"Pipeline timed out after {timeout} seconds.")
            raise TimeoutError(f"Pipeline timed out after {timeout} seconds.")

        # Wait for a short time before checking again
        time.sleep(10) # Adjust the wait time as needed

```

```
# Do something based on the final status
if status == pipeline_state.PipelineState.PIPELINE_STATE_SUCCEEDED:
    logging.info("Pipeline succeeded")
    return True
elif status == pipeline_state.PipelineState.PIPELINE_STATE_CANCELLED:
    logging.error("Pipeline cancelled")
    raise Exception("Pipeline cancelled")
elif status == pipeline_state.PipelineState.PIPELINE_STATE_FAILED:
    logging.error("Pipeline failed")
    raise Exception("Pipeline failed")
```

15. 在新的筆記本儲存格中，執行公用函式來執行管道

```

def run_job_with_psc_interface_config(
    project_id: str,
    region: str,
    pipeline_root: str,
    network_attachment_name: str,
    ip_address: str,
    local_pipeline_file: str = "pipeline.yaml",
):
    """
    Compiles, submits, and monitors a Vertex AI pipeline.
    """

    parameter_values = {"ip_address": ip_address}
    pipeline_root = f"{pipeline_root}/{datetime.now().strftime('%Y%m%d%H%M%S')}"
    logging.info("Compiling pipeline")

    try:
        with open(local_pipeline_file, "r") as stream:
            pipeline_spec = yaml.safe_load(stream)
            logging.info(f"Pipeline Spec: {pipeline_spec}")
    except yaml.YAMLError as exc:
        logging.error(f"Error loading pipeline yaml file: {exc}")
        raise

    logging.info(f"Will use pipeline root: {pipeline_root}")
    # Initialize the Vertex SDK using PROJECT_ID and LOCATION
    aiplatform.init(project=project_id, location=region)

    # Create the API endpoint
    client_options = {"api_endpoint": f"{region}-aiplatform.googleapis.com"}

    # Initialize the PipelineServiceClient
    client = aiplatform_v1beta1.PipelineServiceClient(client_options=client_options)

    # Construct the request
    request = aiplatform_v1beta1.CreatePipelineJobRequest(
        parent=f"projects/{project_id}/locations/{region}",
        pipeline_job=aiplatform_v1beta1.PipelineJob(
            display_name="pipeline-with-psc-interface-config",
            pipeline_spec=pipeline_spec,
            runtime_config=aiplatform_v1beta1.PipelineJob.RuntimeConfig(
                gcs_output_directory=pipeline_root, parameter_values=parameter_values
            ),
            psc_interface_config=aiplatform_v1beta1.PscInterfaceConfig(
                network_attachment=network_attachment_name
            ),
        ),
    )

    # Make the API call
    response = client.create_pipeline_job(request=request)

```

```
# Print the response
logging.info(f"Pipeline job created: {response.name}")

return response.name
```

16. 在新的筆記本儲存格中，編譯管道

```
from kfp import compiler
compiler.Compiler().compile(pipeline_func=pipeline, package_path='pipeline.yaml')
```

17. 在新的筆記本儲存格中，更新 TARGET_IP_ADDRESS，反映先前步驟中為 test-svc-1 取得的 IP 位址，並觀察管道工作狀態

```
TARGET_IP_ADDRESS = "<your-test-svc-1-ip>"
try:
    job_name = run_job_with_psc_interface_config(
        project_id=PROJECT_ID,
        region=REGION,
        pipeline_root=PIPELINE_ROOT,
        network_attachment_name=NETWORK_ATTACHMENT_URI,
        ip_address=TARGET_IP_ADDRESS,
    )
    wait_for_pipeline(
        project_id=PROJECT_ID,
        region=REGION,
        pipeline_job_resource_name=job_name,
    )
except Exception as e:
    logging.error(f"An error occurred: {e}")
```

範例：

```
TARGET_IP_ADDRESS = "192.168.20.2"
try:
    job_name = run_job_with_psc_interface_config(
        project_id=PROJECT_ID,
        region=REGION,
        pipeline_root=PIPELINE_ROOT,
        network_attachment_name=NETWORK_ATTACHMENT_URI,
        ip_address=TARGET_IP_ADDRESS,
    )
    wait_for_pipeline(
        project_id=PROJECT_ID,
        region=REGION,
        pipeline_job_resource_name=job_name,
    )
except Exception as e:
    logging.error(f"An error occurred: {e}")
```

執行步驟 17 後，管道約需 8 分鐘才能完成。

12. 驗證與 test-svc-1 的連線

在用於執行步驟 17 的儲存格中，觀察管道工作狀態從 PIPELINE_STATE_PENDING 轉換為 PIPELINE_STATE_RUNNING，最終轉換為 PIPELINE_STATE_SUCCEEDED，這表示 Vertex AI Pipelines 已成功發出 Ping，且 test-svc-1 已回應。

如要驗證 Vertex AI Pipeline 與 test-svc-1 之間的 ICMP 流量，請查看先前在 test-svc-1 OS 中執行的 tcpdump 工作階段，該工作階段會提供雙向流量的記錄。

在 tcpdump 範例中，Vertex AI Pipelines 是從 192.168.10.0/28 子網路取得 IP 位址 192.168.10.3，而 192.168.20.2 是 test-svc-1 的 IP 位址。請注意，您環境中的 IP 位址可能不同。

```
user@test-svc-1:~$ sudo tcpdump -i any icmp -nn
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
18:57:54.737490 ens4 In IP 192.168.10.3 > 192.168.20.2: ICMP echo request, id 257, seq 0,
length 64
18:57:54.737523 ens4 Out IP 192.168.20.2 > 192.168.10.3: ICMP echo reply, id 257, seq 0,
length 64
```

13. Vertex AI Pipelines AI to test-svc-2 connectivity

在下一節中，您將建立輸入防火牆規則，拒絕從 Vertex AI Pipelines 子網路 (192.168.10.0/28) 到 test-svc-2 的流量，然後更新 Notebook 以反映 test-svc-2 IP 位址，最後執行 Pipelines Jobs 執行。

在 Notebook 儲存格中，管道工作狀態會顯示「錯誤 - 管道失敗」，而防火牆記錄則會提供連線失敗的洞察資料。

建立拒絕輸入防火牆規則

如要拒絕從 Vertex AI Pipelines 連線至 test-svc-2，請建立輸入防火牆規則，將 PSC 網路附件指定為來源 (192.168.10.0/28)，並將 test-svc-2 IP 位址指定為目的地。

在 Cloud Shell 中，更新 destination-range，使其與 test-svc-2 IP 位址相符。

```
gcloud compute --project=$projectid firewall-rules create deny-icmp-vertex-pipelines-to-test-svc2-vm --direction=INGRESS --priority=1000 --network=consumer-vpc --action=DENY --source-ranges=192.168.10.0/28 --rules=ALL --destination-ranges=<your-test-svc-2-vm-ip> --rules=icmp --enable-logging
```

範例：

```
gcloud compute --project=$projectid firewall-rules create deny-icmp-vertex-pipelines-to-test-svc2-vm --direction=INGRESS --priority=1000 --network=consumer-vpc --action=DENY --source-ranges=192.168.10.0/28 --rules=ALL --destination-ranges=192.168.30.2 --enable-logging
```

從 Notebook Cell 執行管道工作

在新的筆記本儲存格中，更新 TARGET_IP_ADDRESS，反映先前步驟中為 test-svc-2 取得的 IP 位址，並觀察 Pipelines 工作狀態。

```
TARGET_IP_ADDRESS = "<your-test-svc-2-ip>"
try:
    job_name = run_job_with_psc_interface_config(
        project_id=PROJECT_ID,
        region=REGION,
        pipeline_root=PIPELINE_ROOT,
        network_attachment_name=NETWORK_ATTACHMENT_URI,
        ip_address=TARGET_IP_ADDRESS,
    )
    wait_for_pipeline(
        project_id=PROJECT_ID,
        region=REGION,
        pipeline_job_resource_name=job_name,
    )
except Exception as e:
    logging.error(f"An error occurred: {e}")
```

範例：

```
TARGET_IP_ADDRESS = "192.168.30.2"
try:
    job_name = run_job_with_psc_interface_config(
        project_id=PROJECT_ID,
        region=REGION,
        pipeline_root=PIPELINE_ROOT,
        network_attachment_name=NETWORK_ATTACHMENT_URI,
        ip_address=TARGET_IP_ADDRESS,
    )
    wait_for_pipeline(
        project_id=PROJECT_ID,
        region=REGION,
        pipeline_job_resource_name=job_name,
    )
except Exception as e:
    logging.error(f"An error occurred: {e}")
```

執行後，管道工作約需 8 分鐘才能完成。

14. 驗證失敗，無法連線至 test-svc-2

在用於執行 Pipeline Job 的儲存格中，請注意狀態從 PIPELINE_STATE_PENDING 轉換為 PIPELINE_STATE_FAILED，這表示 Vertex AI Pipelines 的連線偵測 (ping) 失敗，且 test-svc-2 未回應。

使用記錄檢視器，您可以查看符合「拒絕輸入」規則的防火牆記錄項目，該規則包含 Vertex AI Pipelines 子網路 (192.168.10.0/28) 和 test-svc-2 IP 位址。

選取「顯示查詢」，然後插入下列篩選條件，最後 15 分鐘，然後執行查詢。

```
jsonPayload.rule_details.reference:("network:consumer-vpc/firewall:deny-icmp-vertex-pipelines-to-test-svc2-vm")
```

注意：如果未產生記錄，請重新載入網路瀏覽器，然後使用提供的篩選條件和時間間隔執行「run query」。

The screenshot shows the Google Cloud Logs Explorer interface. At the top, there's a search bar with the query: `jsonPayload.rule_details.reference:("network:consumer-vpc/firewall:deny-icmp-vertex-pipelines-to-test-svc2-vm")`. Below the search bar, there are filters for 'All resources', 'All log names', 'All severities', and 'Correlate by'. A 'Run query' button is visible. The interface shows a timeline view with a time range from Feb 10, 4:33:45 PM to Feb 10, 4:49:00 PM. The timeline shows a single log entry at 4:44 PM. Below the timeline, there's a table with 5 results. The table has columns for SEVERITY, TIME, and SUMMARY. The results show denied connections from Vertex AI Pipelines to test-svc-2.

SEVERITY	TIME	SUMMARY
Warning	2025-02-10 16:41:13.383	{"connection":{...}, "disposition":"DENIED", "instance":{...}, "remote_instance":{...}, "remote_vpc":{...}, "rule_details":{...}, "vpc":{...}}
Warning	2025-02-10 16:41:16.387	{"connection":{...}, "disposition":"DENIED", "instance":{...}, "remote_instance":{...}, "remote_vpc":{...}, "rule_details":{...}, "vpc":{...}}
Warning	2025-02-10 16:41:19.388	{"connection":{...}, "disposition":"DENIED", "instance":{...}, "remote_instance":{...}, "remote_vpc":{...}, "rule_details":{...}, "vpc":{...}}
Warning	2025-02-10 16:41:22.311	{"connection":{...}, "disposition":"DENIED", "instance":{...}, "remote_instance":{...}, "remote_vpc":{...}, "rule_details":{...}, "vpc":{...}}
Warning	2025-02-10 16:41:25.315	{"connection":{...}, "disposition":"DENIED", "instance":{...}, "remote_instance":{...}, "remote_vpc":{...}, "rule_details":{...}, "vpc":{...}}

選取記錄項目，然後展開巢狀欄位，顯示由 Vertex AI Pipelines 和 test-svc-2 IP 位址組成的資訊元素，驗證遭拒絕的輸入防火牆規則。

```
▼ jsonPayload: {
  ▼ connection: {
    dest_ip: "192.168.30.3"
    dest_port: 33437
    protocol: 17
    src_ip: "192.168.10.12"
    src_port: 50132
  }
  disposition: "DENIED"
  ▼ instance: {
    project_id: "psc-vertex"
    region: "us-central1"
    vm_name: "test-svc-2"
    zone: "us-central1-a"
  }
}
```

步驟 15 · 約 10 分鐘

15. 清理

在 Cloud Shell 中刪除教學課程元件。

```
gcloud compute instances delete test-svc-1 test-svc-2 --zone=us-central1-a --quiet

gcloud workbench instances delete workbench-tutorial --location=us-central1-a --quiet

gcloud compute firewall-rules delete deny-icmp-vertex-pipelines-to-test-svc2-vm allow-icmp-vertex-pipelines-to-test-svc1-vm ssh-iap-consumer --quiet

gcloud compute routers nats delete cloud-nat-us-central1 --router=cloud-router-us-central1 --region us-central1 --quiet

gcloud compute routers delete cloud-router-us-central1 --region=us-central1 --quiet
```

步驟 16 · 約 0 分鐘

16. 恭喜

恭喜！您已成功設定並驗證 Private Service Connect 介面，以及消費者和生產者連線，方法是實作 Ingress 允許和拒絕防火牆。

您已建立消費者基礎架構，並新增網路連結，讓 Vertex AI Pipelines 服務建立 PSC 介面 VM，以橋接消費者和生產者通訊。您已瞭解如何在消費者虛擬私有雲網路中建立防火牆規則，允許及拒絕連線至消費者網路中的執行個體。

Cosmopup 認為教學課程很棒！



後續步驟

延伸閱讀和影片

- [Private Service Connect 總覽](#)

What is Private Service Connect?

Google Cloud Tech

Private Service Connect



Watch on YouTube

Vertex AI networking patterns - PSC & Google APIs demo

Advanced Networking Demo

Connect to Vertex AI using PSC & Googleapis



Watch on YouTube

參考文件

- [為管道設定 Private Service Connect 介面 | Vertex AI | Google Cloud](#)
- [為 Vertex AI 資源設定 Private Service Connect 介面 | Google Cloud](#)